

Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет компьютерных систем и сетей

Кафедра электронных вычислительных машин

Дисциплина: Операционные системы и системное программирование

ОТЧЕТ
по лабораторной работе № 2
Тема: «Понятие процессов»

Выполнил:
студ. гр. 450502 Бурш А.П.

Проверил:
пр. Благиных В.Д.

Минск, 2026

1 ЦЕЛЬ ЛАБОРАТОРНОЙ РАБОТЫ

Цель работы – разработать две программы – `parent` (родительский процесс) и `child` (дочерний процесс).

Родительский процесс, запуская дочерний, создает для него сокращенную среду (окружение). Для этого пользователем создается файл `env`, содержащий небольшой набор имён переменных окружения, передаваемых при вызове `execve()`.

Перед запуском `parent` в ее окружении создается переменная `CHILD_PATH` с именем каталога, где находится программа `child`.

Родительский процесс (программа `parent`) после запуска получает переменные своего окружения и их значения, установленные оболочкой, сортирует в `LC_COLLATE=C` и выводит в `stdout`. Читает файл `env` и формирует среду для дочернего процесса в том виде, в котором она указывается в системном вызове `execve()`, используя значения для переменных из собственной среды. После этого входит в цикл обработки нажатий клавиатуры.

Символ «+»

Родительский процесс, используя `fork()` и `execve(2)` порождает дочерний процесс и запускает в нем очередной экземпляр программы `child`. Информацию о каталоге, где размещается `child`, `parent` получает из своего окружения, используя функцию `getenv()`.

Имя программы `child( argv[0] )` устанавливается как `child_XX`, где `XX` – порядковый номер от 00 до 99 (номер инкрементируется родителем). Дочерний процесс выводит свое имя, `pid` и `ppid` в `stdout`. Вторым параметром программы `child` является путь к файлу `env`, который читается дочерним процессом для получения ему переданных значений параметров среды. Дочерний процесс открывает этот файл, считывает имена переменных, получает из окружения их значение, используя `getenv()`, и выводит в `stdout`.

Символ «*»

Дочерний процесс порождается аналогично предыдущему случаю, однако информацию о своём окружении программа `child` получает, сканируя массив параметров среды, переданный в третьем параметре функции `main()` и выводит в `stdout`.

Символ «q»

Завершает выполнение родительского процесса после завершения дочернего.

2 ВЫПОЛНЕНИЕ РАБОТЫ

2.1 Описание алгоритма выполнения работы

Программа состоит из родительского (`parent`) и дочернего (`child`) процессов для работы с переменными окружения. Родительский процесс при запуске выводит отсортированный список своего окружения, затем ожидает команды пользователя («+», «*» или «q»), в зависимости от которой выполняются разные действия. Для хранения переменных используется динамический массив с автоматическим расширением. Каждый дочерний процесс выводит своё имя (в формате `child_XX`), PID и значения переменных окружения. Программа обрабатывает ошибки и корректно освобождает ресурсы.

2.2 Описание основных функций

Программа состоит из нескольких частей, представляющих собой некоторые функции. К ним относятся функции:

- 1) Сортировка и вывод окружения `parent`;
- 2) Формирование сокращенного окружения `child`;
- 3) Запуск `child` через `fork()/execve()`;
- 4) Вывод окружения в `child`;
- 5) Ожидание завершения `child`;
- 6) Освобождение выделенной памяти.

2.2.1 Функция `getEnvp()`

Функция копирует переменные окружения из `envp` в динамически выделенный массив строк; затем в `main` этот массив сортируется через `compStrings()`.

2.2.2 Функция `getChildEnv()`

Функция читает имена переменных из файла, путь к которому задан в `ENV_PATH`, получает значения через `getenv()` и формирует массив строк `NAME=VALUE` для `execve()`.

2.2.3 Функция `executeChild()`

Функция формирует имя `child_XX`, подготавливает аргументы запуска и выполняет `execve()` для программы `child`, `CHILD_PATH` берется из окружения.

2.2.4 Функция `printStrings()`

Функция выводит переданный массив строк окружения, полученный в parent или child через getEnvp().

2.2.5 Функция main()

Функция main() организует цикл команд (+, *, q), запускает child и корректно завершает parent с освобождением памяти.

2.2.6 Функция waitForChild()

Функция waitForChild() ожидает завершение дочернего процесса с помощью waitpid().

2.2.7 Функция freeEnv()

Функция freeEnv() освобождает память, выделенную под массив строк окружения env (например, parentEnv или childEnv).

3 ОПИСАНИЕ ПОРЯДКА СБОРКИ

Проект собирается с помощью CMake. В каталоге проекта выполняются команды cmake -S . -B build и cmake --build build, которые создают исполняемые файлы build/parent и build/child из src/parent/main.c, src/child/main.c и src/environment.c. Для запуска необходимо задать переменные окружения CHILD_PATH (путь к build/child) и ENV_PATH (путь к env.txt), после чего выполнить ./build/parent. Очистка выполняется командой cmake --build build --target clean.

4 РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ

```
Parent environment variables (66):
-----
AQ_DRM_DEVICES=/dev/dri/card1
CHROME_DESKTOP=code.desktop
COLORTERM=truecolor
DBUS_SESSION_BUS_ADDRESS=unix:path=/run/user/1000/bus
DEBUGINFOD_URLS=https://debuginfod.archlinux.org
DESKTOP_SESSION=hyprland
DISPLAY=:1
ENV_PATH=/home/ekuzm/Programming/OSaSP/lab2/env.txt
FC_FONTATIONS=1
GDK_BACKEND=wayland
GIT_ASKPASS=/opt/visual-studio-code/resources/app/extensions/git/dist/askpass.sh
HL_INITIAL_WORKSPACE_TOKEN=396a6286-1982-4687-bcc9-8a4cc4827358
HOME=/home/ekuzm
HYPRCURSOR_SIZE=24
HYPRLAND_CMD=Hyprland --watchdog-fd 4
HYPRLAND_INSTANCE_SIGNATURE=dd220efe7b1e292415bd0ea7161f63df9c95bfd3_1772186636_578709298
HYPRSHOT_DIR=/home/ekuzm/Pictures/Screenshots/
KITTY_INSTALLATION_DIR=/usr/lib/kitty
KITTY_PID=3277
KITTY_PUBLIC_KEY=1:*Z?92c1o^~xSJuf_KItDYNU>vv>&ZIS<@6kEJvUK
KITTY_WINDOW_ID=1
LANG=en_US.UTF-8
LESS=-R
LOGNAME=ekuzm
LSCOLORS=Gxfxcxdxbxegedabagacad
LS_COLORS=rs=0:di=01;34:ln=01;36:mh=00:pi=40;33:so=01;35:do=01;35:bd=40;33;01:cd=40;33;01:or=40;31;01:mi=00:su=37;41:sg=30;43:ca=00:tw=30;42:ow=34;42:st=37;44:ex=01;32:*.7z=01;31:*.ace=01;31:*.alz=01;31:*.apk=01;31:*.arc=01;31:*.arj=01;31:*.bz=01;31:*.bz2=01;31:*.c
```

```

ab=01;31:*.cpio=01;31:*.crate=01;31:*.deb=01;31:*.drpm=01;31:*.dwm=01;31:*.dz=01;31:*.ear=01;31:*.egg=01;31:*.esd=01;31:*.gz=01;31:*.
jar=01;31:*.lha=01;31:*.lrz=01;31:*.lz=01;31:*.lz4=01;31:*.lzh=01;31:*.lzma=01;31:*.lzo=01;31:*.pyz=01;31:*.rar=01;31:*.rpm=01;31:*.rz=01;3
1:*.sar=01;31:*.swm=01;31:*.t7z=01;31:*.tar=01;31:*.taz=01;31:*.tbz=01;31:*.tbz2=01;31:*.tgz=01;31:*.tlz=01;31:*.txz=01;31:*.tz=01;31:*.tz
o=01;31:*.tztz=01;31:*.udeb=01;31:*.war=01;31:*.whl=01;31:*.wim=01;31:*.xz=01;31:*.z=01;31:*.zip=01;31:*.zoo=01;31:*.zst=01;31:*.avif=0
1;35:*.jpg=01;35:*.jpeg=01;35:*.jxl=01;35:*.mjpg=01;35:*.mjpeg=01;35:*.gif=01;35:*.bmp=01;35:*.pbm=01;35:*.pgm=01;35:*.ppm=01;35:*.t
ga=01;35:*.xbm=01;35:*.xpm=01;35:*.tif=01;35:*.tiff=01;35:*.png=01;35:*.svg=01;35:*.svgz=01;35:*.mng=01;35:*.pcx=01;35:*.mov=01;35:*.
mpg=01;35:*.mpeg=01;35:*.m2v=01;35:*.mkv=01;35:*.webm=01;35:*.webp=01;35:*.ogm=01;35:*.mp4=01;35:*.m4v=01;35:*.mp4v=01;35:*.
vob=01;35:*.qt=01;35:*.nuv=01;35:*.wmv=01;35:*.asf=01;35:*.rm=01;35:*.rmvb=01;35:*.flc=01;35:*.avi=01;35:*.fli=01;35:*.flv=01;35:*.gl=01
;35:*.dl=01;35:*.xcf=01;35:*.xwd=01;35:*.yuv=01;35:*.cgm=01;35:*.emf=01;35:*.ogv=01;35:*.ogx=01;35:*.aac=00;36:*.au=00;36:*.flac=00;36
:*.m4a=00;36:*.mid=00;36:*.midi=00;36:*.mka=00;36:*.mp3=00;36:*.mpc=00;36:*.ogg=00;36:*.ra=00;36:*.wav=00;36:*.oga=00;36:*.opus=00
;36:*.spx=00;36:*.xspf=00;36:*.~=00;90:*.bak=00;90:*.crdownload=00;90:*.dpgk-dist=00;90:*.dpgk-new=00;90:*.dpgk-
old=00;90:*.dpgk-
tmp=00;90:*.old=00;90:*.orig=00;90:*.part=00;90:*.rej=00;90:*.rpmnew=00;90:*.rpmorig=00;90:*.rpmsave=00;90:*.swp=00;90:*.tmp=00;90:
*.ucf-dist=00;90:*.ucf-new=00;90:*.ucf-old=00;90:
MAIL=/var/spool/mail/ekuzm
MOTD_SHOWN=pam
MOZ_ENABLE_WAYLAND=1
NO_AT_BRIDGE=1
OLDPWD=/home/ekuzm/Programming/OSaSP/lab2/build
PAGER=less
PATH=/home/ekuzm/.config/Code/User/globalStorage/github.copilot-
chat/debugCommand:/home/ekuzm/.config/Code/User/globalStorage/github.copilot-
chat/copilotCli:/usr/local/bin:/usr/bin:/bin:/usr/local/sbin:/usr/lib/jvm/default/bin:/usr/bin/site_perl:/usr/bin/vendor_perl:/usr/bin/core_perl:
/var/lib/snapd/snap/bin
PWD=/home/ekuzm/Programming/OSaSP/lab2/build
SHELL=/usr/bin/zsh
SHLVL=3
TERM=xterm-256color
TERMINFO=/usr/lib/kitty/terminfo
TERM_PROGRAM=vscode
TERM_PROGRAM_VERSION=1.109.2
USER=ekuzm
USER_ZDOTDIR=/home/ekuzm
VSCODE_GIT_ASKPASS_EXTRA_ARGS=
VSCODE_GIT_ASKPASS_MAIN=/opt/visual-studio-code/resources/app/extensions/git/dist/askpass-main.js
VSCODE_GIT_ASKPASS_NODE=/opt/visual-studio-code/code
VSCODE_GIT_IPC_HANDLE=/run/user/1000/vscode-git-033b0c699d.sock
VSCODE_INJECTION=1
VSCODE_PYTHON_AUTOACTIVATE_GUARD=1
WAYLAND_DISPLAY=wayland-1
XCURSOR_SIZE=24
XDG_BACKEND=wayland
XDG_CURRENT_DESKTOP=Hyprland
XDG_DATA_DIRS=/usr/local/share:/usr/share:/var/lib/snapd/desktop
XDG_RUNTIME_DIR=/run/user/1000
XDG_SEAT=seat0
XDG_SEAT_PATH=/org/freedesktop/DisplayManager/Seat0
XDG_SESSION_CLASS=user
XDG_SESSION_DESKTOP=Hyprland
XDG_SESSION_ID=2
XDG_SESSION_PATH=/org/freedesktop/DisplayManager/Session1
XDG_SESSION_TYPE=wayland
XDG_VTNR=1
ZDOTDIR=/home/ekuzm
----- MENU -----
+ Show env in child via getenv()
* Show env in child via envp[] (main arg)
q Quit
-----
Select option (+, *, q):

```

Рисунок 1 – Запуск parent

```

=====
Child Environment Viewer
=====
Program : child_00
PID   : 313404
PPID  : 313369
-----
/home/ekuzm/Programming/OSaSP/lab2/env.txtCan't find value of HOSTNAME. Key skipped.
Can't find value of LC_COLLATE. Key skipped.

```

```

Child process environment via getenv():
ENV_PATH=/home/ekuzm/Programming/OSaSP/lab2/env.txt
HOME=/home/ekuzm
LANG=en_US.UTF-8
LOGNAME=ekuzm
PATH=/home/ekuzm/.config/Code/User/globalStorage/github.copilot-
chat/debugCommand:/home/ekuzm/.config/Code/User/globalStorage/github.copilot-
chat/copilotCli:/usr/local/bin:/usr/bin:/bin:/usr/local/sbin:/usr/lib/jvm/default/bin:/usr/bin/site_perl:/usr/bin/v
SHELL=/usr/bin/zsh
TERM=xterm-256color
USER=ekuzm

=====
Child Environment Viewer
=====
Program : child_01
PID   : 313920
PPID  : 313369
-----
Child process environment via *envp[]:
ENV_PATH=/home/ekuzm/Programming/OSaSP/lab2/env.txt
HOME=/home/ekuzm
LANG=en_US.UTF-8
LOGNAME=ekuzm
PATH=/home/ekuzm/.config/Code/User/globalStorage/github.copilot-
chat/debugCommand:/home/ekuzm/.config/Code/User/globalStorage/github.copilot-
chat/copilotCli:/usr/local/bin:/usr/bin:/bin:/usr/local/sbin:/usr/lib/jvm/default/bin:/usr/bin/site_perl:/usr/bin/v
SHELL=/usr/bin/zsh
TERM=xterm-256color
USER=ekuzm

----- MENU -----
+ Show env in child via getenv()
* Show env in child via envp[] (main arg)
q Quit
-----
Select option (+, *, q): q

```

Рисунок 2 – вывод child в режимах + и *

ЗАКЛЮЧЕНИЕ

При выполнении лабораторной работы были изучены процессы parent/child и передача переменных окружения в `execve()`.

Реализована программа, которая формирует сокращенную среду дочернего процесса и позволяет сравнить способы чтения окружения через `environ` и `envp`.